

Ligador, Bibliotecas e Ligação Estática

Seminário de Software Básico

Alberto Henrique Gabriel Ramos Ícaro Vinícius Vinícius Macedo

Grupo 2

2026

Contexto

Definições

Processos e Etapas

Vantagens e Desvantagens

Exemplo Prático

Considerações Finais

Contexto

Pipeline de Compilação



O **Linker** é a **última etapa** antes do executável final.

Definições

O que é o Ligador (Linker)?

Definição

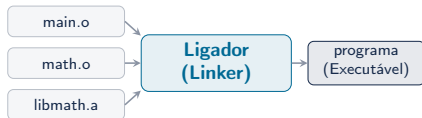
Programa que **combina** múltiplos arquivos-objeto e bibliotecas em um **único executável**.

Entrada: arquivos .o + bibliotecas

Processo: resolução de símbolos +
relocação

Saída: executável binário

Exemplos: `ld` (GNU), `link.exe` (MSVC)



Definição

Coleções de **código pré-compilado** empacotadas para **reutilização**.

Biblioteca Estática

- Extensão: `.a` / `.lib`
- Código **copiado** para o executável
- Criada com `ar rcs`

Biblioteca Dinâmica

- Extensão: `.so` / `.dll`
- Carregada em **runtime**
- **Compartilhada** entre programas

Definição

Processo em que todo o código das bibliotecas é **incorporado** no executável em **tempo de compilação**.

- ✓ Executável **autossuficiente**
- ✓ Não depende de `.so/.dll`
- ✓ Arquivo **maior**, mas **independente**

Ligação Estática

Código do Programa

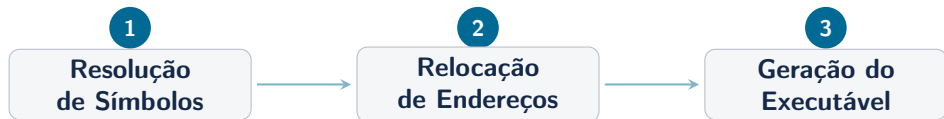
Biblioteca (.a) Incorporada

Ligação Dinâmica

Código do Programa

Referência a Biblioteca (.so)

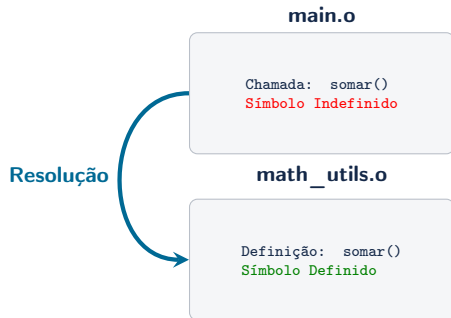
Processos e Etapas



1. **Resolução de Símbolos** — associa cada referência à sua definição
2. **Relocação** — ajusta endereços para o espaço de memória final
3. **Geração** — mescla seções `.text`, `.data`, `.bss` no executável

Como funciona?

- Cada .o tem uma **tabela de símbolos**
- Símbolos **definidos** — funções/variáveis locais
- Símbolos **indefinidos** — referências externas
- Ligador **casa** referências ↔ definições



Erro comum

undefined reference to 'func'

Problema

Cada `.o` assume endereço inicial `0x0`. Quando combinados, os endereços **colidem**.

Antes (endereços relativos)

`main.o` → `.text` em `0x0000`
`math.o` → `.text` em `0x0000`

Depois (endereços absolutos)

`main.o` → `.text` em `0x1000`
`math.o` → `.text` em `0x2000`

O ligador **recalcula** todos os endereços para posições finais no executável.

Arquivo gerado

Formato **ELF** (Linux) ou **PE** (Windows):

- **.text** — código de máquina
- **.data** — variáveis inicializadas
- **.bss** — variáveis não inicializadas
- **Header** — metadados, entry point
- **Symbol Table** — para depuração

ELF Header

.text (código)

.data (dados)

.bss

Tabela de Símbolos

Vantagens e Desvantagens

Vantagens

- **Portabilidade** — sem dependências
- **Desempenho** — sem overhead dinâmico
- **Confiabilidade** — imune ao DLL Hell
- **Distribuição** simples

Desvantagens

- **Tamanho** do executável maior
- **Duplicação** de código em RAM
- **Atualização** — recompilar tudo
- **Memória** — consumo maior

Casos de uso: sistemas embarcados, CLIs, containers Docker, binários Go/Rust

Exemplo Prático

string_utils.h

```
1  #ifndef STRING_UTILS_H
2  #define STRING_UTILS_H
3
4  long long my_strlen(const char *
        str);
5  void my_reverse(char *str);
6
7  #endif
```

main.c

```
1  #include <stdio.h>
2  #include "string_utils.h"
3
4  int main() {
5      char texto[] = "Software
        Basico";
6      long long len = my_strlen(
        texto);
7      printf("Tamanho: %lld\n", len)
        ;
8      my_reverse(texto);
9      printf("Invertido: %s\n",
        texto);
10     return 0;
11 }
```

my_strlen.s

```
1  .globl my_strlen
2  my_strlen:
3      movq $0, %rax
4  .loop:
5      cmpb $0, (%rdi, %rax)
6      je .end
7      incq %rax
8      jmp .loop
9  .end: ret
```

my_reverse.s (Trecho)

```
1  .globl my_reverse
2  my_reverse:
3      movq %rdi, %rsi
4  .find_end:
5      cmpb $0, (%rsi)
6      je .found_end
7      incq %rsi
8      jmp .find_end
9  .found_end:
10     decq %rsi
11     # ... (swap loop e ret)
```

1. Compilacao e Criacao da Biblioteca (.a)

```
1 $ as my_strlen.s -o my_strlen.o
2 $ as my_reverse.s -o my_reverse.o
3 $ ar rcs libstrutils.a my_strlen.o my_reverse.o
```

2. Ligação Estática com o Programa C

```
1 $ gcc -c main.c -o main.o
2 $ gcc main.o -L. -lstrutils -static -o programa
```

Resultado: programa — executável 100% independente.

Execução

```
1 $ ./programa
2 Tamanho: 15
3 Invertido: ocisaB erawtfoS
```

Tipo do arquivo

```
1 $ file programa
2 programa: ELF 64-bit LSB
3 executable, x86-64,
4 statically linked
```

Símbolos da Biblioteca

```
1 $ nm programa | grep -E
2     "my_strlen|my_reverse"
3 00000000004010a0 T my_strlen
4 00000000004010c0 T my_reverse
```

Verificacao com ldd

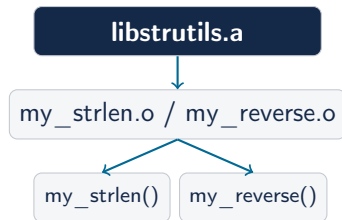
```
1 $ ldd programa
2     not a dynamic executable
```

Conteúdo do Arquivo

```
1 $ ar -t libstrutils.a
2 my_strlen.o
3 my_reverse.o
```

Símbolos Exportados

```
1 $ nm libstrutils.a
2 my_strlen.o:
3 0000000000000000 T my_strlen
4 my_reverse.o:
5 0000000000000000 T my_reverse
```



O arquivo .a empacota os arquivos de objeto.

Considerações Finais

Ligador

Combina .o's
em executável

Biblioteca

Código pré-compilado
para reutilização

Lig. Estática

Incorpora tudo
no executável

- O Linker é **essencial** no pipeline de compilação
- Ligação estática → executáveis **portáteis** e **autossuficientes**
- Trade-off: **tamanho** vs **independência**

- BRYANT, R. E.; O'HALLARON, D. R. *Computer Systems: A Programmer's Perspective*. 3ª ed. Pearson, 2015. Cap. 7.
- LEVINE, J. R. *Linkers and Loaders*. Morgan Kaufmann, 1999.
- GNU Binutils Documentation. <https://www.gnu.org/software/binutils/>
- TANENBAUM, A. S. *Sistemas Operacionais Modernos*. 4ª ed. Pearson, 2016.

Perguntas?

Obrigado pela atenção!

Grupo 2 — Alberto, Gabriel, Ícaro e Vinícius